COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	07 / 30 / 2026	**DATE SUBMITTED:**	
NAME:	Elijah I. Dizor	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4.1 Stacks

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Implement a stack using the C++ STL

CODE

ILO A.cpp Untitled2

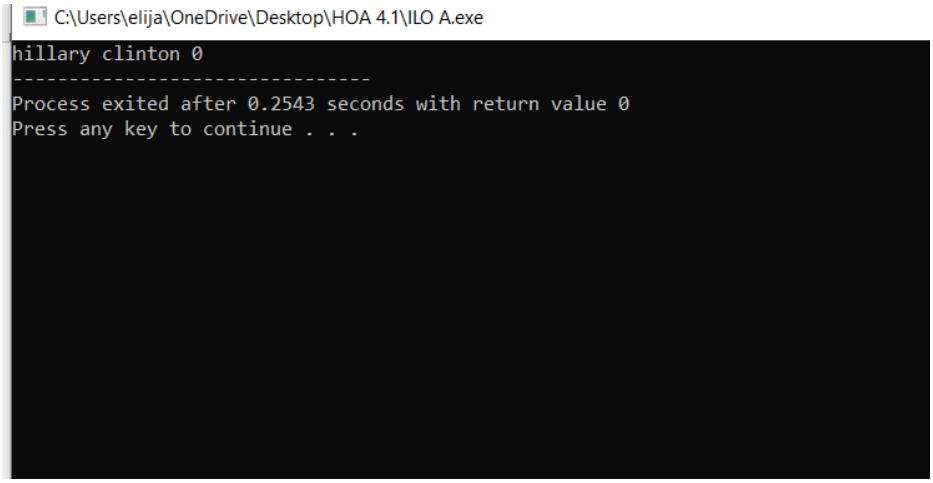
```cpp
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<string> characters;

    characters.push("issa rei");
    characters.push("sydney sweedney");
    characters.push("hillary clinton ");
    characters.push("paige");
    characters.pop();

    cout << characters.top();

    cout<< characters.empty();
    return 0;
}
```

OUTPUT	
OBSERVATION	Based on the code, the last string pushed onto the stack 'paige' is removed by the pop() function. It prints 'hillary clinton', because it becomes the new top lament.

ILO B.1: Implement a stack using a Static Array.

CODE	ILO B1.cpp
	1 #include <iostream>
	2
	3 const size_t maxCap = 100;
	4 int stack[maxCap];
	5 int top = -1, newData, elements;
	6
	7 //prototyping
	8 bool isFull();
	9 bool isEmpty();
	10 void push();
	11 void pop();
	12 void Top();
	13
	14 int main(){
	15 std::cout << "Enter the number of maximum elements for the new stack (max: 100): ";
	16 std::cin >> elements;
	17
	18 if (elements > maxCap) {
	19 elements = maxCap;
	20 }
	21
	22 while(true){
	23 int choice = 0;
	24 std::cout << "\nSelect Stack Operation:" << std::endl;
	25 std::cout << "1. PUSH\n2. POP\n3. VIEW TOP\n4. IS EMPTY?\n5. ISFULL" << std::endl;
	26 std::cin >> choice;

ILO B1.cpp

```
28          switch(choice){
29              case 1:
30                  push();
31                  break;
32
33              case 2:
34                  pop();
35                  break;
36
37              case 3:
38                  Top();
39                  break;
40
41              case 4:
42                  if (isEmpty()) {
43                      std::cout << "The stack is empty (true)." << std::endl;
44                  } else {
45                      std::cout << "The stack is not empty (false)." << std::endl;
46                  }
47                  break;
48
49              case 5:
50                  if (isFull()) {
51                      std::cout << "The stack is full (true)." << std::endl;
52                  } else {
53                      std::cout << "The stack is not full (false)." << std::endl;
54                  }
55                  break;
56
57              default:
58                  std::cout << "Invalid Choice" << std::endl;
59                  break;
60          }
```

ILO B1.cpp

```
62      return 0;
63  }
64
65  bool isEmpty(){
66      return (top == -1);
67  }
68
69  bool isFull(){
70      return (top >= elements - 1);
71  }
72
73  void push(){
74      if (isFull()){
75          std::cout << "Stack Overflow! Cannot push more items." << std::endl;
76          return;
77      }
78      std::cout << "What number would you like to add? ";
79      std::cin >> newData;
80      stack[++top] = newData;
81  }
82
83  void pop(){
84      if (isEmpty()){
85          std::cout << "Stack Underflow! Cannot pop from an empty stack." << std::endl;
86          return;
87      }
88      std::cout << "Popping: " << stack[top] << std::endl;
89      top--;
```

```
}

void Top(){
    if(isEmpty()){
        std::cout << "Stack is empty! No top element." << std::endl;
        return;
    }
    std::cout << "The top element is " << stack[top] << std::endl;
}
```

OUTPUT

```
Select C:\Users\elija\OneDrive\Desktop\HOA 4.1\ILO B1.exe
Enter the number of maximum elements for the new stack (max: 100): 47

Select Stack Operation:
1. PUSH
2. POP
3. VIEW TOP
4. IS EMPTY?
5. ISFULL
1
What number would you like to add? 32

Select Stack Operation:
1. PUSH
2. POP
3. VIEW TOP
4. IS EMPTY?
5. ISFULL
2
Popping: 32

Select Stack Operation:
1. PUSH
2. POP
3. VIEW TOP
4. IS EMPTY?
5. ISFULL
3
Stack is empty! No top element.

Select Stack Operation:
1. PUSH
2. POP
3. VIEW TOP
4. IS EMPTY?
5. ISFULL
4
The stack is empty (true).

Select Stack Operation:
1. PUSH
2. POP
3. VIEW TOP
4. IS EMPTY?
5. ISFULL
5
The stack is not full (false).

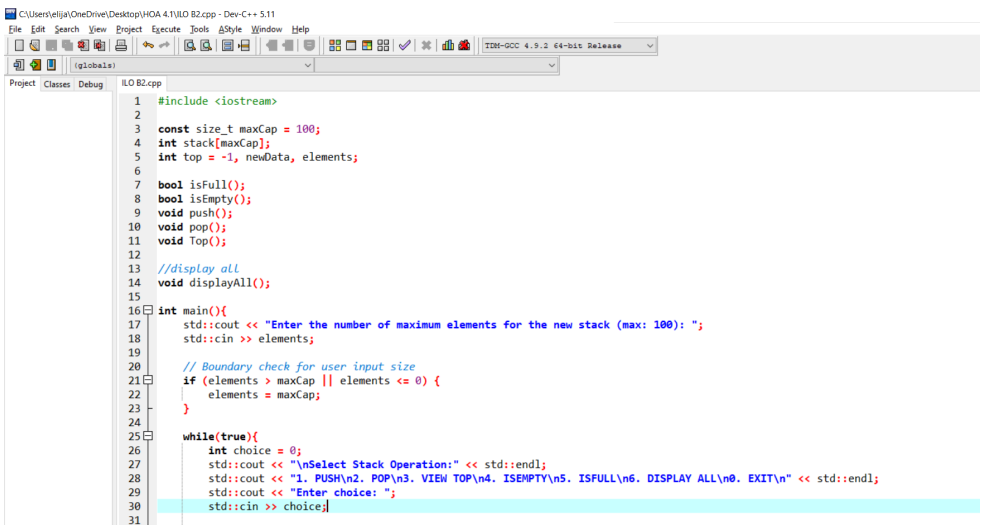
Select Stack Operation:
1. PUSH
2. POP
```

OBSERVATION

Based on the code, the program prompts the user to define the stack capacity defined by maxCap. A boundary check ensures that if the input exceeds the array limit, as the variable element is automatically capped at 100 to prevent out-of-bounds array access. The while and if loop statements, paired with a ‘switch case’ block runs continuously

ILO B.2 Stacks using Linked Lists

CODE



```cpp
#include <iostream>

const size_t maxCap = 100;
int stack[maxCap];
int top = -1, newData, elements;

bool isFull();
bool isEmpty();
void push();
void pop();
void Top();

//display all
void displayAll();

int main(){
    std::cout << "Enter the number of maximum elements for the new stack (max: 100): ";
    std::cin >> elements;

    // Boundary check for user input size
    if (elements > maxCap || elements <= 0) {
        elements = maxCap;
    }

    while(true){
        int choice = 0;
        std::cout << "\nSelect Stack Operation:" << std::endl;
        std::cout << "1. PUSH\n2. POP\n3. VIEW TOP\n4. ISEMPTY\n5. ISFULL\n6. DISPLAY ALL\n0. EXIT\n" << std::endl;
        std::cout << "Enter choice: ";
        std::cin >> choice;
```

ILO B2.cpp

```cpp
        switch(choice){
            case 1:
                push();
                break;

            case 2:
                pop();
                break;

            case 3:
                Top();
                break;

            case 4:
                if (isEmpty()) {
                    std::cout << "Stack is empty (true)" << std::endl;
                } else {
                    std::cout << "Stack is not empty (false)" << std::endl;
                }
                break;

            case 5:
                if (isFull()) {
                    std::cout << "Stack is full (true)" << std::endl;
                } else {
                    std::cout << "Stack is not full (false)" << std::endl;
                }
                break;

            case 6:
                displayAll();
                break;
```

ILO B2.cpp

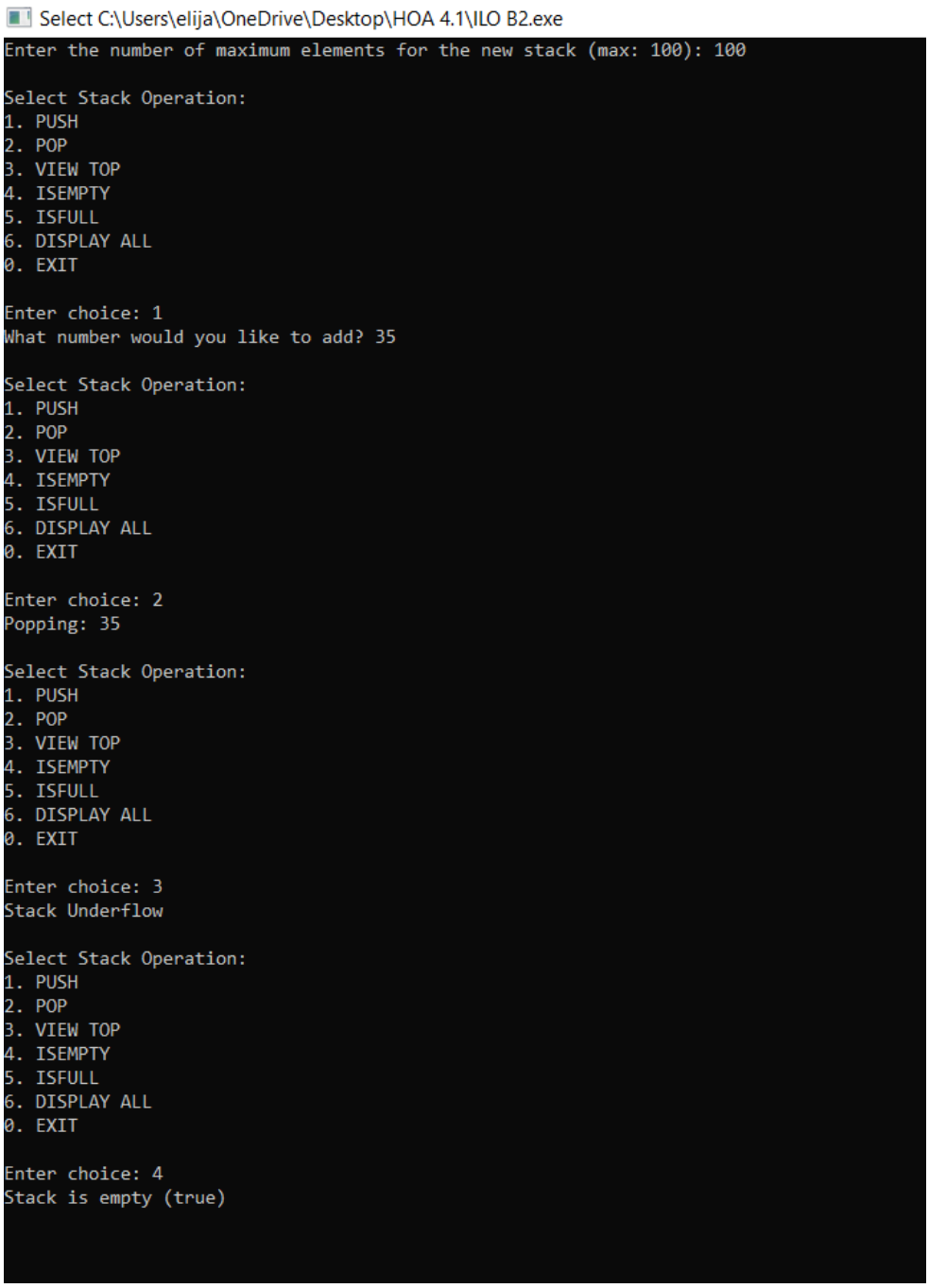
```
65              case 0:
66                  std::cout << "Exiting program..." << std::endl;
67                  return 0;
68
69              default:
70                  std::cout << "Invalid Choice" << std::endl;
71                  break;
72          }
73      }
74
75      return 0;
76  }
77
78  bool isEmpty(){
79      //top is -1
80      if(top == -1){
81          return true;
82      }
83      return false;
84  }
85
86  bool isFull(){
87      //not equal to maxCap
88      if(top >= elements - 1){
89          std::cout << "Stack Overflow" << std::endl;
90          return true;
91      }
92      return false;
93  }
```

ILO B2.cpp

```
92      return false;
93  }
94
95  void push(){
96      //check if stack is full, if yes, return error
97      if(isFull()){
98          return;
99      }
100     //ask user input values to be pushed into stack
101     std::cout << "What number would you like to add? ";
102     std::cin >> newData;
103     //store inside stack
104     stack[++top] = newData;
105 }
106
107 void pop(){
108     // check if empty
109     if (isEmpty()){
110         std::cout << "Stack Underflow" << std::endl;
111         return;
112     }
113
114     //display element to be popped
115     std::cout << "Popping: " << stack[top] << std::endl;
116
117     //decrement top:
118     top--;
119 }
```

```cpp
void top(){
    //error checking
    if(isEmpty()){
        std::cout << "Stack Underflow" << std::endl;
        return;
    }
    //display top
    std::cout << "The top element is: " << stack[top] << std::endl;
}

void displayAll(){
    if (isEmpty()){
        std::cout << "Nothing to display!" << std::endl;
        return;
    }
    std::cout << "Stack elements (top to bottom): ";
    for(int i = top; i >= 0; i--){
        std::cout << stack[i] << " ";
    }
    std::cout << std::endl;
}
```

OUTPUT	
OBSERVATION	The program implements a standard stack using an integer array defined by stack, to which users can define a custom stack capacity 'elements' up to a strict cap depending on what limit they prefer. The push, pop, and top functions manipulate the top index tracking variable, and the displayAll function prints out the stack elements starting from the 'top' to index 0.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C: Solve problems using an implementation of stack:

CODE	ILO C.cpp
	1 #include <iostream>
	2 #include <string>
	3
	4 using namespace std;
	5
	6 struct Node {
	7 char data;
	8 Node* next;
	9 };
	10
	11 class Stack {
	12 private:
	13 Node* top;
	14
	15 public:
	16 Stack() { top = nullptr; }
	17
	18 void push(char val) {
	19 Node* newNode = new Node();
	20 newNode->data = val;
	21 newNode->next = top;
	22 top = newNode;
	23 }

ILO C.cpp

```
24
25      char pop() {
26          if (isEmpty()) {
27              return '\0';
28          }
29          Node* temp = top;
30          char poppedValue = temp->data;
31          top = top->next;
32          delete temp;
33          return poppedValue;
34      }
35
36      bool isEmpty() {
37          return top == nullptr;
38      }
39
40      ~Stack() {
41          while (!isEmpty()) {
42              pop();
43          }
44      }
```

ILO C.cpp

```
    }
};

bool isBalanced(string expr) {
    Stack s;

    for (int i = 0; i < expr.length(); i++) {
        char ch = expr[i];

        if (ch == '(' || ch == '[' || ch == '{') {
            s.push(ch);
        }

        else if (ch == ')' || ch == ']' || ch == '}') {

            if (s.isEmpty()) {
                return false;
            }

            char openSymbol = s.pop();

            if ((ch == ')' && openSymbol != '(') ||
                (ch == ']' && openSymbol != '[') ||
                (ch == '}' && openSymbol != '{')) {
                return false;
            }
        }

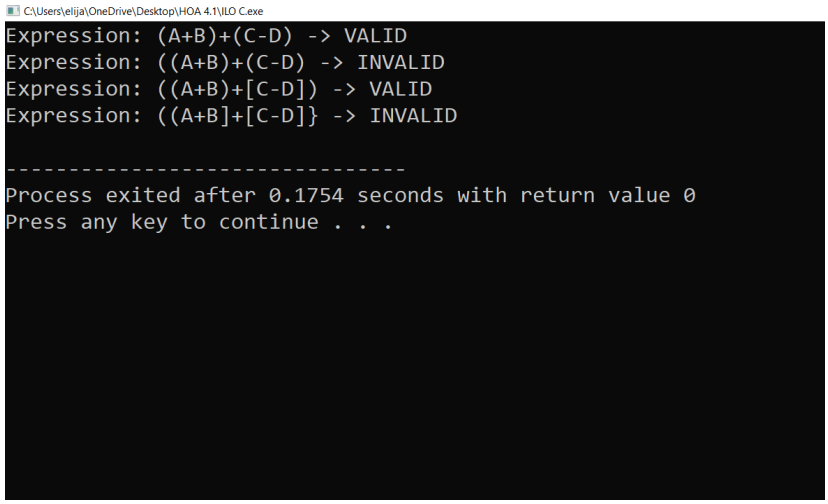
    }

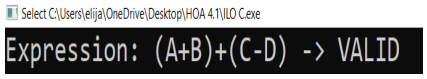
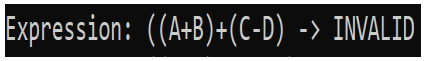
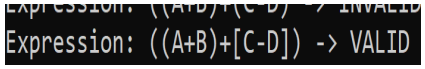
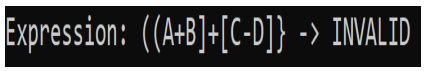
    return s.isEmpty();
}

int main() {
    string expressions[] = {
        "(A+B)+(C-D)",
        "((A+B)+(C-D)",
        "((A+B)+[C-D])",
        "((A+B]+[C-D]}"
    };

    for (const string& expr : expressions) {
        cout << "Expression: " << expr << " -> "
             << (isBalanced(expr) ? "VALID" : "INVALID") << endl;
    }

    return 0;
}
```

OUTPUT	
OBSERVATION	Based on the code, it implements a stack using singly linked list and the use of custom 'Node' struct. It allocates memory using 'new' for stack nodes to avoid fixed size limits. The isBalanced function scans the strings line-by line, pushing open brackets onto the stack and popping them.

Expression	Valid? (Y/N)	Output(Console screenshot)	Analysis
(A+B)+(C-D)	Y		All opening brackets match their corresponding closing brackets, so the stack is empty at the end
((A+B)+(C-D)	N		Two opening brackets but only one closing bracket. One unmatched brackets so it remains in the stack
((A+B)+[C-D])	Y		Both parenthesis and square brackets open and close in the correct order, so the stack finishes empty
(([A+B]+[C-D])	N		The expression contains a mismatched bracket paris, a bracket or

			braces attempts to close a parenthesis

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

"I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own."